ROS 2 / MANIPULATOR CONTROL / HARDWARE-SOFTWARE INTEGRATION

myCobot 280 Pi Control System

Modular joint and Cartesian interfaces for a 6-DOF robot and virtual model



Bench setup with the physical cobot, external computing hardware, and RViz visualization.

Project objective

Create flexible ROS 2 command paths for a physical myCobot 280 Pi and its virtual model, supporting interactive sliders and script-defined targets while migrating control across onboard and external computers.

My contribution

Developed ROS 2 interfaces, launch configurations, physical-robot communication, script publishers, and a MoveIt 2 IK client. Integrated a pre-existing controller GUI and worked through Raspberry Pi, external PC, and Jetson hardware configurations.

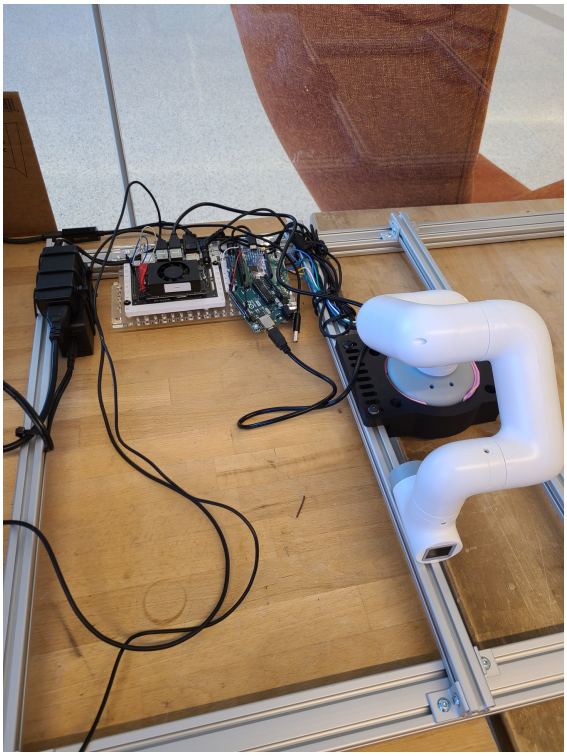
Python | ROS 2 | MoveIt 2 | RViz | UART | pymycobot | URDF/Xacro

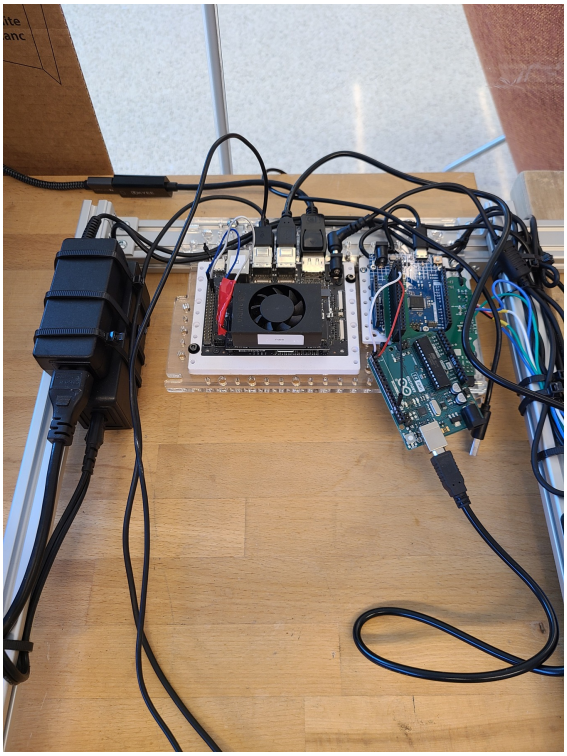
Dakshitha (Dak) Amarasinghe

dakshithaama1994@gmail.com | LinkedIn profile

01 / PLATFORM MIGRATION

Adapting the control platform





Author-supplied bench photos showing the cobot, external compute, and wired interfaces. Migration history below follows the supplied resume description.

1. Onboard computing upgrade

Replaced the original Raspberry Pi 4B with a Raspberry Pi 5 as an initial control-platform upgrade.

2. External PC control

Moved primary control to an external PC and established UART communication to the auxiliary ESP32 using a CH340 USB-to-TTL converter.

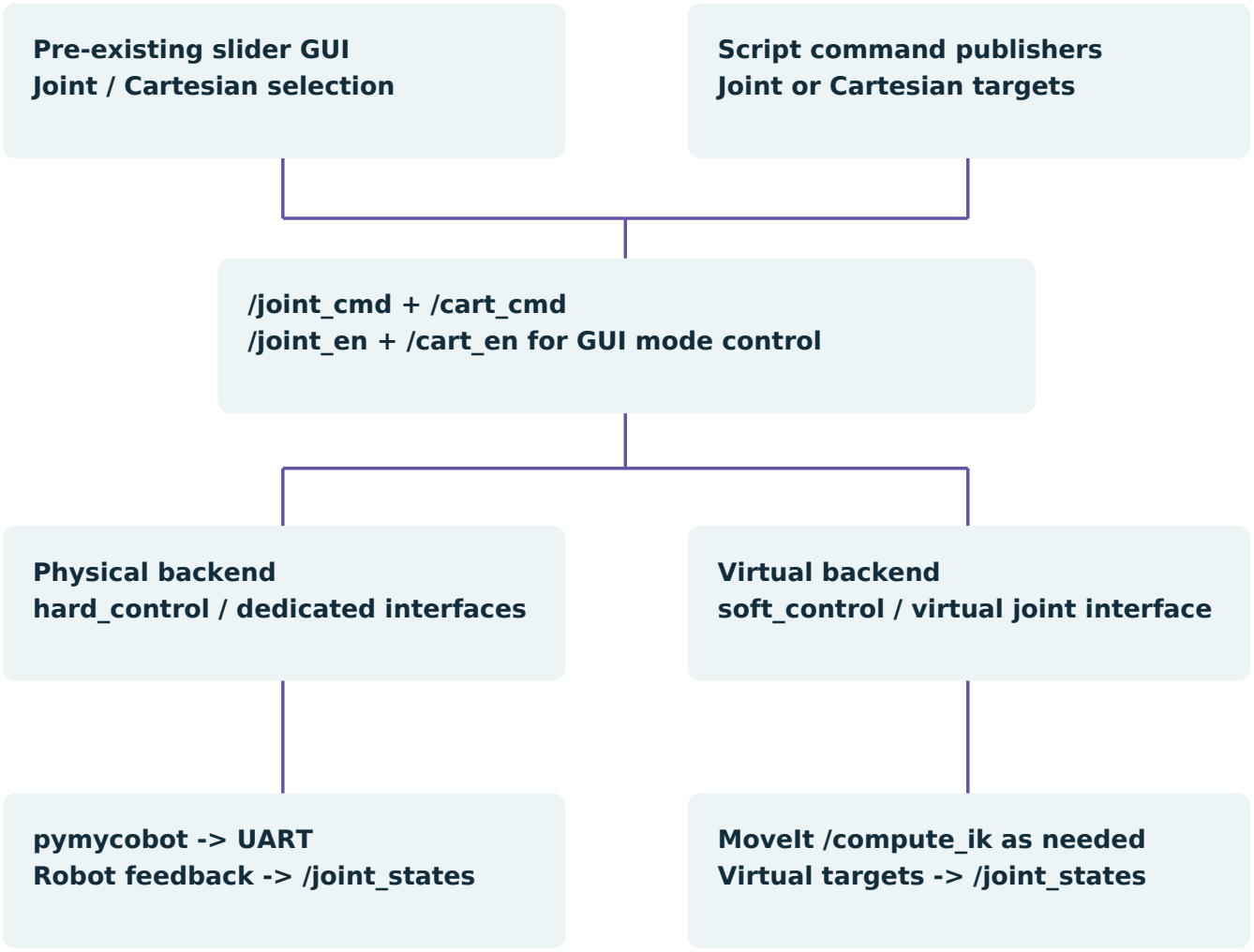
3. Jetson migration and bridge

Migrated control to an NVIDIA Jetson Orin Nano. When the CH340 converter was not recognized, implemented an Arduino Uno UART bridge to restore the communication path.

Engineering focus: separate host-platform compatibility from the ROS command interfaces and robot-side communication. The migration narrative is author-reported; no Arduino bridge firmware is included in the supplied workspace snapshot.

02 / MODULAR ROS 2 DESIGN

Shared topics, separate backends



robot_state_publisher + URDF -> TF -> RViz robot visualization

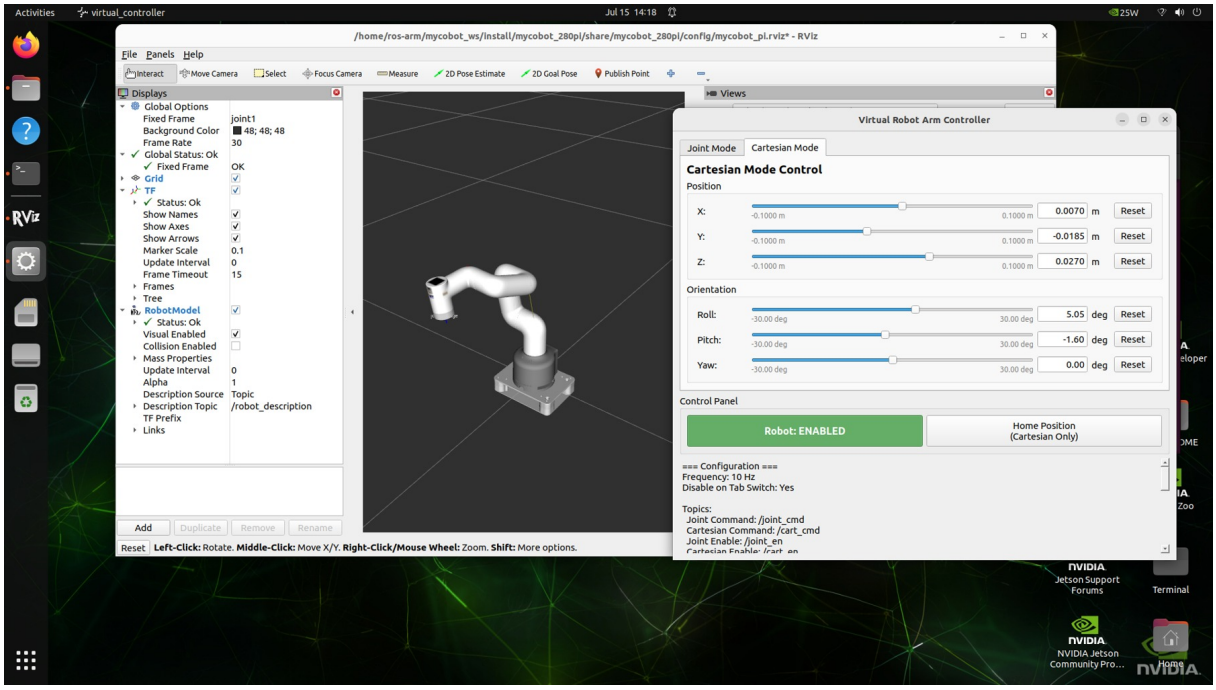
Independent workflows

Hardware and virtual controllers are alternative launch paths. Each can publish /joint_states, so backend selection matters; the screenshots do not establish a continuously synchronized digital twin.

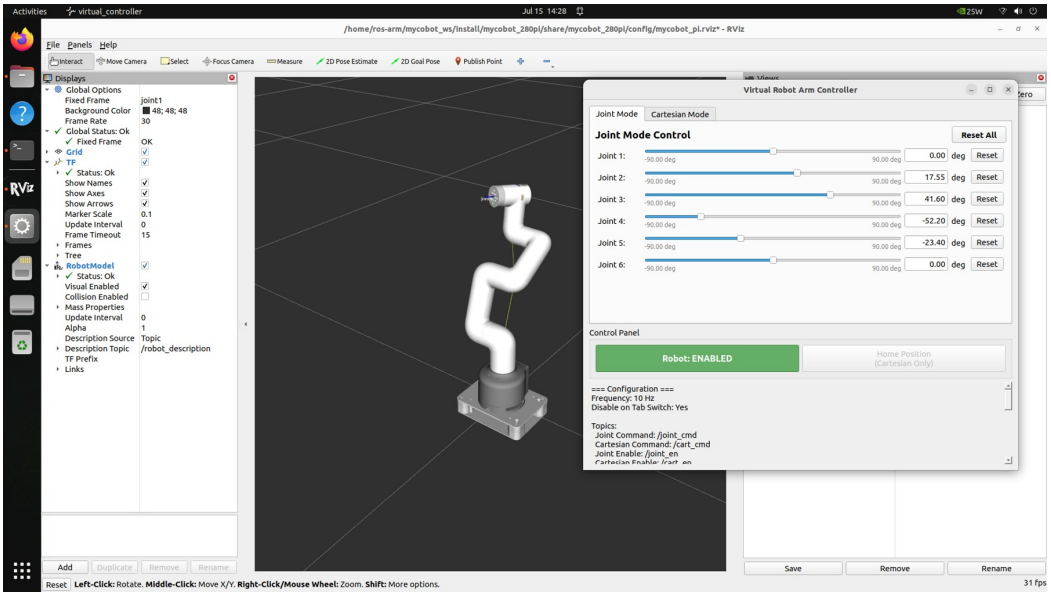
The diagram summarizes multiple launch variants. Not every workflow needs every topic or node. The script IK node can also translate /cart_cmd into /joint_cmd for the selected backend.

03 / INTERACTIVE CONTROL

Interactive slider control



Supplied Dak1 screenshot: slider controller beside the RViz robot model.



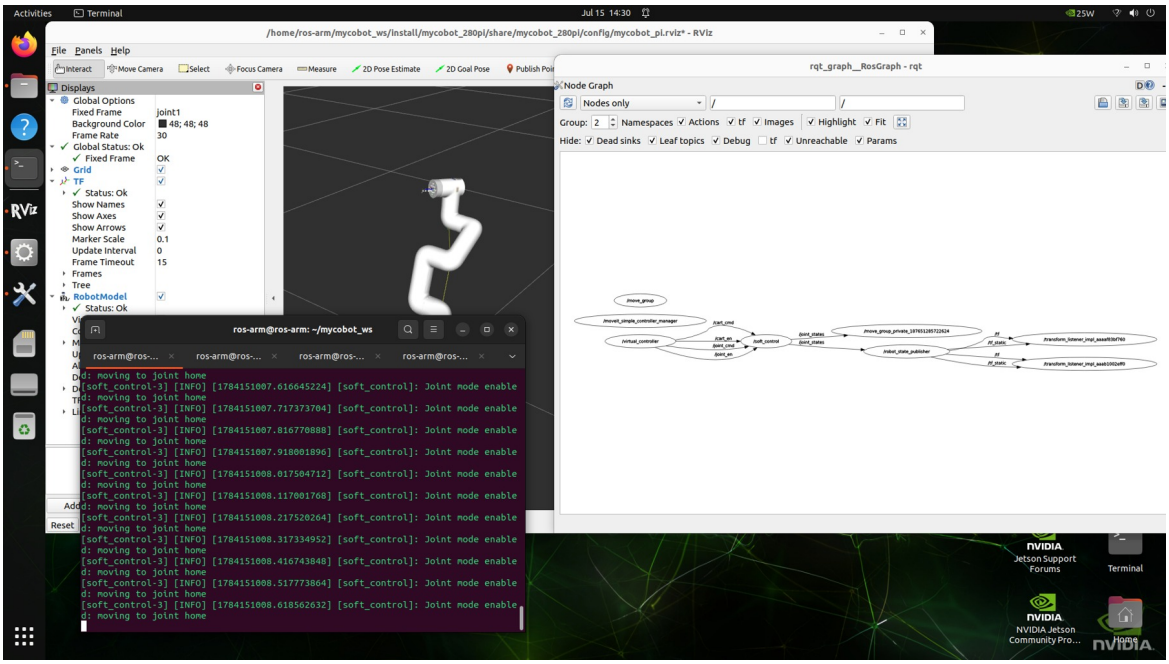
Supplied Dak3 screenshot: an alternate model pose with joint-angle slider controls.

Interface contribution

Connected the pre-existing GUI to ROS 2 control nodes and launch files. Joint targets arrive on /joint_cmd as six values in radians; the hardware interface converts them to degrees for the robot API.

04 / POSE-TO-JOINT CONVERSION

Cartesian targets through MoveIt 2



Supplied Dak4 screenshot: RViz, ROS graph, and command/service logging during development.

The inverse-kinematics pipeline

The custom IK client receives /cart_cmd, combines position and roll/pitch/yaw offsets with a reference pose, converts Euler orientation to a quaternion, and sends a PoseStamped target to MoveIt GetPositionIK at /compute_ik.

Service request and response handling

Requests use frame g_base, planning group arm_group, a joint-state seed, and a one-second timeout setting. Responses are mapped by joint name and reordered before publishing six joint targets on /joint_cmd.

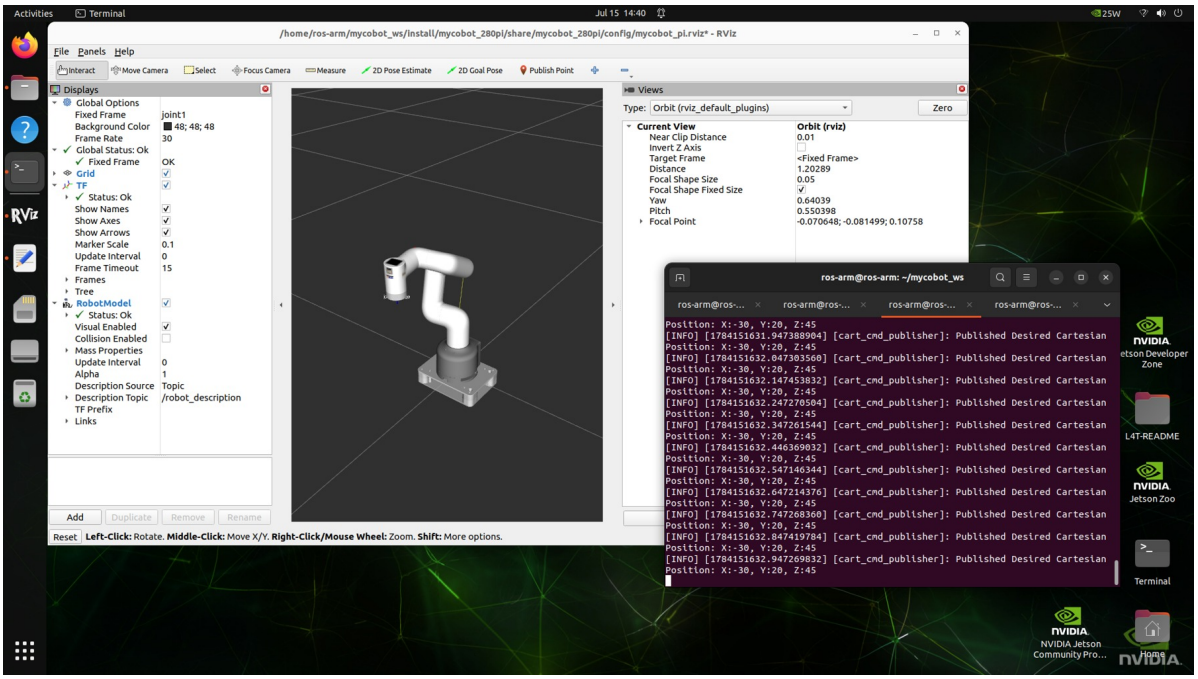
Manage command load

The standalone IK node uses a 0.1-second request interval and an in-flight flag. It avoids requesting a solution for changes below its configured tolerance; these are code settings rather than measured solve-rate guarantees.

Implementation boundary: this is IK target generation, not a complete time-parameterized trajectory planner. The submitted IK requests set avoid_collisions=False; collision-aware execution is not demonstrated by these requests.

05 / REPEATABLE COMMAND INPUT

Script-defined command inputs



Supplied Dak7 screenshot: robot visualization and terminal output in a command-driven workflow.

Joint command publisher

des_jstate_publisher.py exposes a user-editable six-joint target and publishes it through /joint_cmd. The selected physical or virtual joint interface consumes that command.

Cartesian command publisher

des_cstate_publisher.py publishes positional offsets relative to a defined home pose. Its example uses millimeter inputs converted to meters, with angular fields in radians; the supplied example keeps orientation offsets at zero.

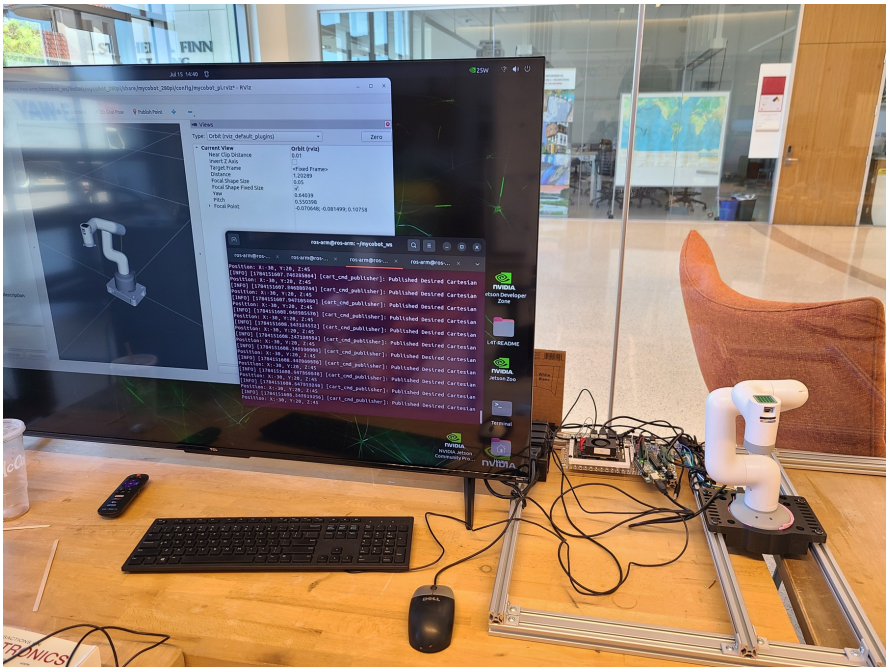
DOCUMENTED LAUNCH WORKFLOW

Hardware: ros2 launch rover_launch mycobot_280pi.launch.py
Virtual: ros2 launch rover_launch mycobot_280pi_vir.launch.py
Joint input: ros2 run mycobot_interface goal_joint_state_pub
Cartesian input: run ik_solver, then goal_cart_state_pub

Launch the GUI separately where required. Hardware access, ROS environment, and MoveIt configuration must match the selected platform.

06 / COMMUNICATION & FEEDBACK

Physical robot control and feedback



Bench photograph: physical robot alongside visualization and terminal activity.

Joint commands and feedback

The hardware node opens a configurable serial connection at a default 1,000,000 baud, checks that joint commands contain six values, converts radians to degrees, and sends angle targets using pymycobot.

Direct Cartesian control variant

The hardware Cartesian path records a reference pose and applies position/orientation offsets. It converts meters to millimeters and radians to degrees, wraps angular targets, and limits command dispatch using a 0.2-second interval.

Communication handling

A lock serializes robot API access. Angle reads retry up to three times and validate a six-element response before publishing /joint_states in radians. The feedback timer is configured at 0.1 seconds.

Rates are configured timer or dispatch intervals, not measured end-to-end performance. The code includes blocking waits, so no hard real-time behavior is claimed.

07 / ENGINEERING ASSESSMENT

Reusable robot-control interfaces

What the project demonstrates

Modular physical and virtual robot-control paths; joint and Cartesian command interfaces; GUI integration; script-defined targets; asynchronous MoveIt IK requests; unit and joint-name conversion; serial communication; and RViz visualization.

Development outcome

The supplied code, launch instructions, hardware photos, and screenshots show an integrated development workflow spanning real hardware and virtual-model operation. The author reports improved responsiveness and smoother motion; no measured latency, trajectory-error, or repeatability benchmark was supplied.

Next engineering improvements

Benchmark command-to-motion latency, serial turnaround, and tracking error. Add systematic command/limit validation and completion feedback. Replace blocking waits where appropriate and evaluate collision-aware planning before broader physical execution.

My work and reused components

Custom work centers on mycobot_interface nodes, rover_launch configurations, hardware migration, and integrating a pre-existing GUI. The robot description, vendor ROS packages, pymycobot, MoveIt 2, and controller GUI provide reused functionality.

Evidence sources

Prepared from the supplied resume description, five hardware photographs, eight screenshots, RvizCommands.txt, and the nested ROS 2 source snapshot. Key reviewed files: hard_control.py, soft_control.py, ik_solver.py, robot_joint_interface.py, robot_cart_interface.py, script publishers, and launch variants.

Review method: static source and image inspection. No physical robot, ROS runtime, or hardware bridge was executed for portfolio preparation. RViz model visualization is not evidence of a physics simulation or safety validation.